Final Project Protocol – TourPlanner

Team Members: Batuhan Saimler, Jansen Wu

Git Repository: https://github.com/disguisedcoder/TourPlanner/tree/new-master

1. Introduction

The goal of the TourPlanner project was to design and implement a fully functional desktop application that allows users to plan, document, and analyze different kinds of tours such as hiking, cycling, or vacation trips. The application supports the complete lifecycle of tour management: from creation, storage, and modification of tours, to recording logs of completed activities, generating statistics, and producing detailed reports.

The project was developed in JavaFX with a PostgreSQL database for persistence, making use of modern software architecture principles such as layered architecture and the MVVM pattern. A central requirement was the integration of external services such as OpenRouteService for distance and route calculation, as well as the inclusion of multiple software engineering best practices: automated testing, logging, and the application of design patterns.

Our application goes beyond the minimum requirements by implementing additional features and polishing the user experience. In the following sections, we describe the technical decisions, design process, and implementation details that shaped our final product.

2. Application Architecture

We structured our application into three distinct layers, each with a clear responsibility. This separation of concerns made it easier to maintain, test, and extend the system over time:

1. UI Layer – Implemented with JavaFX and FXML files, responsible for rendering the user interface and handling direct interactions.
2. Business Layer – Contains the services that implement the domain logic, perform validation, manage CRUD operations, and coordinate persistence.
3. Data Access Layer – Responsible for communication with the PostgreSQL database. This layer was implemented using Hibernate (JPA), with repositories encapsulating all SQL queries.

Key components include domain entities (Tour, TourLog), repositories (TourRepository, TourLogRepository), services (TourService, LogService, ReportService), and ViewModels

(TourListViewModel, SearchBarViewModel, etc.). The class diagram shows the relations between these components.

3. Use Cases

The main use cases of the application include:
- Creating, editing, and deleting tours.
- Managing logs for each tour (add, modify, delete).
- Performing full-text search across tours and logs.
- Importing and exporting data in JSON format.
- Generating PDF reports for individual tours and summaries.

The use case diagram shows the user as the central actor, connected to all of these functionalities. As an example, the sequence diagram for the search process demonstrates how the user query flows through the UI, is processed by the ViewModel and services, and finally retrieves data from the repositories before updating the UI.

4. User Experience and Wireframes

Wireframes were created to guide the development of the user interface. The main window is divided into two panes: a list of tours on the left side and details with logs on the right. Dialogs were designed for creating and editing tours and logs, ensuring clean separation of tasks and preventing clutter in the main interface.

During the development process, the wireframes were iteratively refined. For example, log entry dialogs were made more compact, and confirmation prompts were added for critical actions such as deletions. This iterative approach improved usability significantly.

5. Libraries and Technologies

Several external libraries and frameworks were used to implement required features:
- JavaFX for the UI.
- Hibernate (JPA) for object-relational mapping with PostgreSQL.
- PostgreSQL as the database engine.
- OpenRouteService API for distance and time calculations.
- Leaflet for map visualization.
- log4j for logging.
- Jackson for JSON serialization and deserialization.
- iText for PDF report generation.
- JUnit 5 for unit testing.

These technologies were chosen for their stability, community support, and compatibility with the project goals.

6. Lessons Learned

We encountered multiple challenges during development and learned valuable lessons:

- Implementing MVVM with JavaFX properties required discipline, but once mastered, it simplified data binding and UI updates.
- Hibernate provided a powerful abstraction for persistence but needed careful configuration to avoid lazy-loading issues.
- Integrating the OpenRouteService API required error handling for network failures and rate limits.
- Introducing structured logging early greatly simplified debugging.
- User experience improved after iterative testing and feedback, leading us to refine layouts and dialogs.

7. Implemented Design Patterns

We applied several design patterns to improve code structure and maintainability:

- MVVM (Model-View-ViewModel) pattern for the user interface.
- Repository pattern to encapsulate data access.
- Observer pattern through JavaFX property bindings for reactive UI updates.
- Singleton pattern for configuration management and logging utilities.

These patterns contributed to a cleaner architecture and facilitated testing.

8. Unit Testing Decisions

We implemented more than 20 JUnit tests, focusing on critical functionality:

- CRUD operations for tours and logs.
- Import and export of JSON data.
- Report generation.
- API integration using mocked responses.

By testing these areas, we ensured data integrity, correct behavior of core features, and resilience against faulty inputs or external errors.

9. Unique Feature

We didn’t do one, because we tried to focus on the main part.

10. Time Tracking

We tracked our time throughout the project:

- Planning & Architecture: ~12 hours
- UI Implementation (JavaFX, FXML): ~25 hours
- Business Logic & Database Integration: ~30 hours
- API Integration (OpenRouteService, Leaflet): ~8 hours
- Report Generation & Import/Export: ~7 hours
- Testing & Debugging: ~15 hours
- Documentation & Protocol Writing: ~8 hours

Total: ~105 hours, divided between both members.

11. Git Repository

https://github.com/disguisedcoder/TourPlanner/tree/new-master

12. Conclusion

The TourPlanner project was a comprehensive exercise in software engineering, combining modern UI development, database management, external API integration, and reporting features. By following a layered architecture and the MVVM pattern, we built an application that is modular, extensible, and maintainable. The final product not only fulfills the mandatory requirements but also includes refinements and a unique feature that enhances the user experience. The lessons learned will strongly influence our future projects.